

Sürüm 1.0

C++20

AVX2 / AVX-512 / CUDA

NexusOptim

Ayrıntılı Kılavuz

Kütüphanenin nasıl çalıştığı, her API'nin ve her parametrenin ne işe yaradığı, optimizier formüllerinin eleman eleman hesabı, içeride yapılan hız işleri, ölçülmüş benchmark'lar ve kopyalanabilir örnekler.

Bu belge ikinci baskıdır. Birinci PDF özetti. Bu baskı kaynak koddaki `options` alanlarını, SIMD/CUDA kararlarını ve sayısal yürüyüşleri tek tek açar.

Tek giriş: `#include <nexus_optim/nexus_optim.hpp>`



İçindekiler

1. Bu kütüphane eğitim döngüsünün neresinde durur
2. Bir adım, bellek üzerinde neye dokunur
3. Hız için kodda ne yapıldı
4. Benchmark'lar: nasıl ölçüldü, sayılar ne der
5. Kurulum ve ilk çağrı
6. Ortak API: grup, adım, sıfırlama, durum
7. Öğrenme hızı ile weight decay'in iki yeri
8. SGD, momentum, Nesterov — parametre parametre ve sayısal örnek
9. Adagrad, RMSProp, Adadelta
10. Adam ve AdamW — kapalı formun elle hesabı
11. NAdam, RAdam, AdaBelief
12. LAMB, LARS, Lion
13. Lookahead
14. Yedi zamanlayıcının her argümanı
15. Gradyan fonksiyonları
16. FP16, BF16, CUDA, MatrixFlash
17. Hazır reçeteler
18. Hatalar ve kontrol listesi

1. Bu kütüphane eğitim döngüsünün neresinde durur

Nexus yığnında her kütüphanenin tek işi vardır. NexusOptim'in işi parametre güncellemesidir. Aşağıdaki işler onun dışındadır ve bu başlıklarda fonksiyon olarak da yoktur:

İstenen iş	Sahibi	NexusOptim ne görür
Matris çarpımı, broadcast, GPU GEMM	MatrixFlash Pro	Yalnızca <code>float*</code> ve uzunluk. <code>matmul</code> çağrılmaz.
Veri kümesi, batch, karıştırma	MatrixData / NexusData	Hiçbir şey. Batch optimizer'a girmez.
Katman tanımı, <code>forward</code>	NexusModel	Modelin sahip olduğu parametre işaretçileri.
Kayıp ve <code>backward</code>	NexusLoss	Dolu gradyan tamponu.
Epoch döngüsü, dosya, günlük	NexusTrain	<code>zero_grad</code> , <code>step</code> , <code>scheduler.step</code> , isteğe bağlı <code>state_dict</code> .

Bir adımın doğru sırası sabittir. Sırayı bozmak sayısal sonucu bozar, derleyici hatası vermez:

1. `zero_grad()` — biriktirme yapmıyorsanız. Mikro-batch biriktiriyorsanız bunu N mikro-batch'te bir kez çağırırsınız.
2. İleri geçiş ve geri yayılım. Bu sırada gradyan tamponuna ekleme yapılır. Optimizer henüz çalışmaz.
3. İsteğe bağlı `gradient_centralization_`, sonra `clip_grad_norm_` veya `scale_gradients_`.
4. `optimizer.step()` — parametre değişir, momentler güncellenir, gradyana dokunulmaz.
5. `scheduler.step()` — bir sonraki adımın öğrenme hızını yazar. Bu adımın hızını değiştirmez.

`step()` kayıp değeri görmez. Plateau zamanlayıcısı dışında hiçbir optimizer "loss iyileşti mi" diye sormaz. Loss'u siz tutar, isterseniz `ReduceLROnPlateau::step(loss)` ile iletirsiniz.

2. Bir adım, bellek üzerinde neye dokunur

```
sizin vektörünüz / MatrixFlash tensörü
param[i] — okunur ve yerinde yazılır
grad[i] — yalnız okunur (kırpma fonksiyonları dışında)

optimizer'ın kendi tamponu (kurucuda bir kez ayrılır)
SGD düz : ek tampon yok
momentum / Nesterov : hız v
Adagrad : kümülatif kare s
RMSProp : kare ortalama, isteğe bağlı momentum ve merkez
Adadelat :  $E[g^2]$  ve  $E[\Delta^2]$ 
Adam / AdamW : m, v, isteğe bağlı v_max
NAdam / RAdam : m, v
AdaBelief : m, s
LAMB : m, v, geçici u (kayda girmez)
LARS : isteğe bağlı hız
Lion : yalnızca m
Lookahead : yavaş kopya
FP16 / BF16 : ayrıca FP32 master parametre ve FP32 gradyan çevirisi
```

İşaretçiler ödünçtür. Optimizer'ı yok etmeden parametre vektörünü `std::vector` atamasıyla değiştirirseniz (`g = std::vector<float>(...)`) işaretçi eski belleğe düşer. Yerinde yazın: `g[i] = ...` veya `std::fill`.

Grup sayısı eğitim boyunca sabit kalmalıdır. Zamanlayıcı, `learning_rate` alanının adresini kurucuda kaydeder. `param_groups().push_back` vektörü taşırsa o adres geçersiz olur.

3. Hız için kodda ne yapıldı

Hedef, gradyan hazır olduktan sonra parametreyi bellek bant genişliğine yakın sürede güncellemektir. Bunun için sıcak yolda şunlar bilinçli olarak yapılmadı: sanal fonksiyon, `std::function`, `new`, geçici tensör, moment güncellemesiyle

parametre yazmasının ayrı döngülere bölünmesi.

3.1 Derleme zamanı seçim

`step()` şöyle yazılır: `static_cast<Derived*>(this)->step_impl()`. Bu, Curiously Recurring Template Pattern'dir. Çağrı, derleyici için düz bir fonksiyon çağrısıdır. Hangi optimizier olduğu çalışma anında bir vtable'dan okunmaz. Zamanlayıcılar sanal tutuldu; onlar milyonlarca elemana değil, adım başına bir kez koşar.

Kesinlik de şablondur. `if constexpr (std::is_same_v<ScalarT, float>)` ile double ve float yolları aynı kaynakta durur ama kullanılan yolun karşıtı koda hiç girmez. FP16 depolamada hesap FP32 master üzerinde yürür.

3.2 Katsayılar döngünün dışında

Adam'da `1 - beta1^t` bir kez, `double` olarak hesaplanır ve çekirdeğe `inv_bc1` olarak girer. Eleman döngüsünde `pow` yoktur. Aynı şey NAdam'ın μ çarpımı, RAdam'ın p düzeltmesi ve Adagrad'ın sönmüş öğrenme hızı için geçerlidir. Çekirdek yalnızca toplama, çarpma, karekök ve bölme görür.

3.3 Tek geçiş ve fused çekirdek

Naif Adam dört bellek turu ister: m 'yi yaz, v 'yi yaz, düzeltmeyi oku, parametreyi yaz. NexusOptim bunları tek döngüde yapar. Bir eleman için m , v ve p kayıta üretilir, ara sonuç DRAM'e indirilmez. CUDA tarafında da aynı füzyon vardır: ayrı `add` ve `sqrt` çekirdekleri yerine tek `adam_vec4_kernel`.

3.4 SIMD

Yol	Ne işler	Not
AVX2	8 float veya 4 double	Adam'da $4\times$ açılım: bir turda 32 float. Yükleme <code>_mm256_loadu_ps</code> , yani çağırının hızası şart değildir. FMA kullanılır.
AVX-512	16 float, SGD ve Adam	Derlenir. CPUID AVX-512F ve işletim sisteminin ZMM iznini görmezse çağrılmaz. Bu makinede çağrılmadı.
NEON	4 float, SGD ve Adam	Yalnızca ARM derlemesi. Windows'ta fonksiyon skaler yola düşer ve hiç seçilmez.
Skaler	1 eleman	Kuyruk (8'e bölünmeyen sonlar) ve <code>force_simd_level(Scalar)</code> . MSVC'de <code>#pragma loop(no_vector)</code> ile otomatik vektörleşme kapatıldı ki "skaler" ölçüm gerçekten skaler olsun.

Durum tamponları 64 bayta hizalıdır (`AlignedBuffer`, Windows'ta `_aligned_malloc`). Kullanıcı tensörü hizasız olabilir; bu yüzden kullanıcı verisinde hizalı yük dayatılmaz. Bizim tamponun hızası, işlemci önbellek satırının başından okumayı kolaylaştırır.

3.5 İş parçacığı

`kParallelGrain = 50000`. Bunun altındaki tensörlerde uyandırma, kazançtan pahalıdır; çağırın iş parçacığı tek başına biter. Üstünde süreç boyunca yaşayan bir havuz devreye girer. Havuz optimizier kurucusuna ilk dokunuşta açılır, yani ilk `step()` iş parçacığı yaratmaz.

Bölme 64 elemanın katına yuvarlanır. Böylece komşu iş parçacıkları aynı önbellek satırına yazmaz (false sharing). İşçiler bir adımın ardından kısa süre döner, yeni adım gelmezse uyur. Böylece ardışık eğitim adımları her seferinde işletim sisteminden uyanmak zorunda kalmaz.

LAMB ve LARS bu bölmeyi "her parça kendi güven oranını hesaplasın" diye kullanmaz. Moment güncellemesi paraleldir, kareler toplanır, güven oranı tek sayıdır, ardından uygulama yine paraleldir. İki geçiş sıralıdır; havuz kendini iç içe çağırmaz.

3.6 CUDA

- `sm_86` (RTX 3070) için `-O3` ile derlenir.
- Izgara, akış çok işlemcisi sayısının 32 katına kadar çıkar. Grid-stride döngüsü sabit ızgarayla her uzunluğu tarar.
- SGD, Adam ve Lion'da iş parçacığı 4 float birden taşır. Sonunda 4'e bölünmeyen kuyruk 32 iş parçacıklı skaler çekirdeğe kalır.

- Dört non-blocking stream vardır. `step()` bitmeden stream senkronize edilir; fonksiyon döndüğünde parametre okunabilir. Bu senkron, 1 milyon eleman gibi küçük işlerde sürenin büyük kısmıdır.
- LAMB/LARS normu önce warp içinde, sonra blok içinde toplanır, bloğun tek temsilcisi `atomicAdd(double)` yapar. Eleman başına atomik yoktur.

4. Benchmark'lar

Makine ve komut

Windows, Release, 15 yardımcı iş parçacığı. CPU AVX2 görür, AVX-512 çalıştırmaz. GPU NVIDIA GeForce RTX 3070 Laptop, 8 GB, compute capability 8.6. CUDA 13.4, mimari 86, çekirdek -03.

```
build\benchmarks\Release\bench_adam.exe 1000000 40
build\benchmarks\Release\bench_sgd.exe
build-cuda\benchmarks\Release\bench_cpu_vs_gpu.exe
```

AdamW "28 bayt/eleman" şöyle sayılır: parametre okunur ve yazılır (8), gradyan okunur (4), m okunur ve yazılır (8), v okunur ve yazılır (8). SGD "12 bayt"tır: parametre 8, gradyan 4. AMSGrad veya weight decay ek yük bindirmez çünkü aynı satırlar zaten okunuyordur. Süre, ısınma adımından sonraki turların ortalamasıdır.

Sayılar

İş	Yol	Süre	Hesaplanan bant
AdamW, 1 000 000	skaler, çok iş parçacığı	1712 µs/adım	16.4 GB/s
AdamW, 1 000 000	AVX2	899 µs/adım	31.1 GB/s, skalere göre 1.90×
AdamW, 8 000 000	skaler	12.4 ms/adım	18.1 GB/s
AdamW, 8 000 000	AVX2	9.3 ms/adım	24.1 GB/s, skalere göre 1.33×
SGD, 1 000 000	AVX2	176 µs/adım	68.2 GB/s
AdamW, 1 048 576	AVX2, CUDA'lı ikili	769 µs/adım	çok çekirdek
AdamW, 1 048 576	RTX 3070, stream senkronlu	1168 µs/adım	yaklaşık 25 GB/s etkin

Bu tabloyu şöyle okuyun. SGD karekök ve bölme içermez, bu yüzden aynı makinede AdamW'den belirgin hızlıdır ve 68 GB/s'e çıkar. AdamW'de her eleman `sqr` ve bölme bekler; 8 milyon elemanda iki CPU yolu da belleğe dayanır, AVX2'nin kazancı 1.33×'e iner. 1 milyonluk GPU adımı çekirdekten çok "iş kuyruğa koy ve bitmesini bekle" maliyetidir. Milyarlarca parametrede aynı fused çekirdek bant genişliğine oturur; bu laptop ölçümünde o boy koşturulmadı çünkü momentlerle birlikte onlarca gigabayt gerekir.

"GPU her zaman daha hızlıdır" bu kütüphane için doğru cümle değildir. Küçük ve orta parametre sayısında, senkronlu bir elementwise adımda AVX2 önde olabilir. GPU'yu parametreler zaten cihazda duruyorsa ve tensör büyükse seçin. Her adımda host'a kopyalamak ölçümü anlamsız kılar.

Doğruluk benchmark'ı ayırdır: 40 test geçer. SIMD ile skaler formül 2e-5 bağlı, CUDA ile CPU 2e-4 bağlı, kapalı form SGD/Adam elle hesaplanan değerle 1e-5 içinde örtüşür. Ölçülen boyılar 7, 8, 17, 1024, 10007 ve ayrıca 200 000 elemanlık paralel aralıktır.

5. Kurulum ve ilk çağrı

```
cmake -S . -B build
cmake --build build --config Release
build\examples\Release\basic_sgd_example.exe
build\examples\Release\train_loop_with_nexus_optim.exe
build\tests\Release\nexus_optim_tests.exe
```

CMake seçeneği	Varsayılan	Ne işe yarar
NEXUS_OPTIM_WITH_CUDA	OFF	Açıkken .cu bağlanır ve Device::CUDA adımı çalışır. Kapalıyken CUDA tensörü açık hata verir, CPU kodu hiç CUDA istemez.
CMAKE_CUDA_ARCHITECTURES	86	Hedef GPU. 3070 için 86 doğrudur. Başka kartta nvidia-smi compute capability değerini yazın.
NEXUS_OPTIM_ENABLE_AVX512	ON	OFF ise AVX-512 nesnesi üretilmez, dağıtım AVX2'de durur.
NEXUS_OPTIM_BUILD_TESTS	ON	GoogleTest indirilir ve nexus_optim_tests kurulur.
NEXUS_OPTIM_BUILD_BENCHMARKS	ON	bench_sgd, bench_adam, bench_cpu_vs_gpu.
NEXUS_OPTIM_WARNINGS_AS_ERRORS	ON	Kütüphane C++ dosyalarında uyarı derlemeyi kırar. CUDA çevirisine uygulanmaz.
NEXUS_MATRIXFLASH_INCLUDE	boş	Doluya adaptör başlığının aradığı matrix_pro/ yolu eklenir.

```
cmake -S . -B build-cuda -DNEXUS_OPTIM_WITH_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES=86
cmake --build build-cuda --config Release
build-cuda\tests\Release\nexus_optim_tests.exe --gtest_filter=Cuda.*
```

6. Ortak API

Bütün somut sınıflar OptimizerBase<Türev, ScalarT> üzerinden aynı yüzeyi verir. Şablon yazılmazsa float olur. SGD<> ile SGD<float> aynı tiptir. Kopya yoktur, taşıma vardır.

Çağrı	Ne işe yarar	Ne yapmaz
step()	Bir güncelleme. Momentleri ve parametreyi yazar.	Gradyanı sıfırlamaz. Loss okumaz. Ayırma yapmaz.
zero_grad(bool set_to_none = true)	Gradyan tamponunu bayt bayt sıfırlar. CUDA'da cudaMemset.	İşaretçiyi null yapamaz; bellek sizindir. Bayrak PyTorch çağrı uyumu içindir, iki mod da sıfırlar.
param_groups()	Grup vektörü. Zamanlayıcı learning_rate alanına yazar. Kırpma fonksiyonları bunu ister.	Vektörü büyütmezin.
step_count()	Bitmiş adım sayısı. İlk step sonrası 1. Bias düzeltmesi bu sayıyı kullanır.	Zamanlayıcının epoch'u değildir. Onlar ayrı sayaç tutar.
state_dict()	Algoritma adı, adım, grup hızları, moment baytları.	Parametreyi saklamaz. Parametreyi model checkpoint'i saklar.
load_state_dict	Momentleri ve sayacı geri yazar.	Tensör yerleşimi aynı olmalıdır. Adam kaydı AdamW'ye yüklenmez.

detected_simd_level(), simd_level_name, version() tanıdır. force_simd_level(SimdLevel::Scalar) test içindir; desteklenmeyen yolu zorlamak false döner. std::nullopt algılanan yola geri döner.

7. Öğrenme hızı ile weight decay'in iki yeri

Bu, kütüphanede en çok karıştırılan noktadır. İki ayrı sayı vardır ve adım ikisini birden okur.

Yer	Alan	Adımda etkisi
Grup	options.learning_rate	O grubun η değeri. Zamanlayıcı bunu değiştirir. Asıl hız budur.
Grup	options.weight_decay	Yalnızca weight_decay_override == true iken λ olur.
Grup	weight_decay_override	false (varsayılan): λ , optimizer Options::weight_decay içinden gelir. true: gruptaki λ , sıfır bile olsa, kazanır.
Optimizer	Options::lr	Tek başına parametreyi ölçeklemez. RAdam ve Adagrad gibi formüllerde baz olarak da okunur. Yine de grup hızını aynı değere koyun.
Optimizer	Options::weight_decay	Override olmayan grupların λ değeri.

```
nexus_optim::ParamGroupOptions govde;
govde.learning_rate = 1e-4; // gövde yavaş öğrensin
// override false:  $\lambda$  = AdamW Options içindeki 0.01

nexus_optim::ParamGroupOptions norm;
norm.learning_rate = 1e-4;
norm.weight_decay = 0.0;
norm.weight_decay_override = true; // bias ve LayerNorm'a decay yok

nexus_optim::AdamW<float>::Options opt;
opt.lr = 1e-4;
opt.weight_decay = 0.01;
```

Yalnızca `opt.lr = 1e-3` yazıp grupta `learning_rate` bırakmak, grubun varsayılanı da `1e-3` olduğu için şans eseri çalışır. Grubu `1e-4` yapıp Options'ı `1e-3` bırakırsanız adım `1e-4` kullanır. İkisini bilerek aynı tutun. Zamanlayıcı daha sonra grup alanını değiştirir; Options'taki lr sabit kalır.

8. SGD, momentum, Nesterov

Üç sınıf tek çekirdektir. Fark, kurucuda seçilen momentum ve Nesterov bayrağıdır.

```
g ← g + λ p   eğer λ ≠ 0
eğer μ ≠ 0: v ← μ v + (1 - τ) g
  Nesterov ise g ← g + μ v, değilse g ← v
p ← p - γ g
```

Alan	SGD	MomentumSGD	NesterovSGD	Ne işe yarar
lr	1e-3	1e-3	1e-3	Options kopyası. Asıl γ gruptadır. Çok büyükse adım zıplar, çok küçükse öğrenme durur.
momentum μ	0	0.9	0.9	Eski hızın ne kadarı kalsın. 0 düz SGD'dir, tampon ayrılmaz. 0.9 tipiktir. 1'e yaklaştıkça hız birikir, overshoot artar.
dampening τ	0	0	0	Yeni gradyanın hız eklenen katsayısını (1-τ) yapar. 0 iken gradyan tam eklenir. PyTorch'taki dampening ile aynıdır.
weight_decay λ	0	0	0	L2. Gradyana λp ekler. Parametreyi sıfıra çeker. AdamW'deki ayrık decay değildir.
nesterov	false	false	true	true iken güncelleme hızın bir adım ötesindeki gradyanı kullanır. Momentum 0 iken etkisizdir.

Sayısal yürüyüş — düz SGD

$p = 1, g = 0.1, \gamma = 0.1, \lambda = 0$. Güncelleme: $1 - 0.1 \cdot 0.1 = 0.99$. Üçlü örnek (1, 2, 3) ve (0.1, -0.2, 0.5) için sonuç (0.99, 2.02, 2.95).

$\lambda = 0.5, g = 0, p = 1, \gamma = 0.1$: etkin gradyan $0.5 \cdot 1 = 0.5$, $p \leftarrow 1 - 0.1 \cdot 0.5 = 0.95$. Decay, gradyan sıfırken bile parametreyi küçültür.

Sayısal yürüyüş — momentum

$p = 1, g = 0.5, \gamma = 0.1, \mu = 0.9, v_0 = 0$. İlk adım $v = 0.5$, $p = 1 - 0.1 \cdot 0.5 = 0.95$. İkinci adımda g hâlâ 0.5 ise $v = 0.9 \cdot 0.5 + 0.5 = 0.95$, $p = 0.95 - 0.1 \cdot 0.95 = 0.855$. Hız birikiyor; aynı gradyan ikinci adımda daha sert iter.

Sayısal yürüyüş — Nesterov

Aynı sayılar, ilk adım. $v = 0.5$. Lookahead gradyanı $= 0.5 + 0.9 \cdot 0.5 = 0.95$. $p = 1 - 0.1 \cdot 0.95 = 0.905$. Düz momentumdan daha büyük bir ilk adım atar.

```
nexus_optim::ParamGroupOptions pg;
pg.learning_rate = 0.1;
nexus_optim::MomentumSGD<float>::Options o;
o.lr = 0.1;
o.momentum = 0.9;
nexus_optim::MomentumSGD<float> sgd(
    {nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
sgd.zero_grad();
// backward g'yi doldurur
sgd.step();
```

9. Adagrad, RMSProp, Adadelata

Adagrad — her parametrenin kendi biriken ölçęęi

$$\gamma_t = \gamma / (1 + (t-1) \cdot \text{lr_decay})$$

$$g \leftarrow g + \lambda p$$

$$s \leftarrow s + g^2$$

$$p \leftarrow p - \gamma_t \cdot g / (\sqrt{s} + \epsilon)$$

Alan	Varsayılan	Ne işe yarar
lr	1e-2	Başlangıç hızı. Seyrek özelliklerde (embedding) klasik seçimdir. Zamanla s büyüdüğü için etkin adım kendiliğinden küçülür.
lr_decay	0	Ek olarak γ'yı adım sayısıyla böler. 0 iken yalnızca s'nin büyümesi yavaşlatır.
eps	1e-10	Sıfıra bölmeyi engeller. İlk adımda s = g ² olduğu için payda en az g kadardır.
weight_decay	0	Gradyana eklenen λ. Ayrık değildir.

Sayı: p = 0, g = 1, γ = 0.1, ε = 1e-8, decay 0. s = 1, p ← -0.1 / (1 + 1e-8) ≈ -0.1. Aynı koordinat defalarca gradyan görürse s şişer ve adım küçülür. Bu, seyrek güncellenen satırların büyük adım atmasını, sık güncellenenlerin sakinleşmesini sağlar. Uzun eğitimde s hiç küçülmediği için öğrenme erken kilitlenebilir; o yüzden dil modellerinde yerini AdamW almıştır.

```
nexus_optim::Adagrad<float>::Options o;
o.lr = 0.1; o.eps = 1e-8;
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 0.1;
nexus_optim::Adagrad<float> ada({nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
```

RMSProp — Adagrad'ın unutan hali

$$v \leftarrow \alpha v + (1-\alpha) g^2$$

$$\text{payda} = \sqrt{v} + \epsilon$$

$$\text{centered ise } \text{payda} = \sqrt{(\max(0, v - \bar{g}^2)) + \epsilon}, \quad \bar{g} \text{ de aynı } \alpha \text{ ile ortalanan}$$

$$\mu \neq 0 \text{ ise } \text{tampon} \leftarrow \mu \cdot \text{tampon} + g / \text{payda}, \quad p \leftarrow p - \gamma \cdot \text{tampon}$$

$$\text{değilse } p \leftarrow p - \gamma \cdot g / \text{payda}$$

Alan	Varsayılan	Ne işe yarar
lr	1e-2	Adım büyüklüğü. 1e-3 ile 1e-2 arası yaygındır.
alpha	0.99	Kare geçmişin unutmaya hızı. 1'e yakınsa eski gradyanlar uzun hatırlanır, v yavaş değişir. Küçültürseniz adaptasyon sınırlı olur.
eps	1e-8	Payda tabanı.
momentum	0	0 değilse ikinci bir tampon ayrılır ve güncelleme o tampon üzerinden yapılır.
centered	false	true ise varyans, kare ortalamadan ortalamanın karesi düşülerek hesaplanır. Ek tampon ister.
weight_decay	0	Gradyana λp.

Adadelta — öğrenme hızını kendisi üretir

$$\begin{aligned}E[g^2] &\leftarrow \rho E[g^2] + (1-\rho) g^2 \\ \Delta &\leftarrow g \cdot \sqrt{(E[\Delta^2] + \epsilon) / (E[g^2] + \epsilon)} \\ E[\Delta^2] &\leftarrow \rho E[\Delta^2] + (1-\rho) \Delta^2 \\ \rho &\leftarrow \rho - \gamma \Delta\end{aligned}$$

Alan	Varsayılan	Ne işe yarar
<code>lr</code>	1	Δ 'nın ölçeği. Algoritma birimini kendisi kurduğu için 1 bırakmak PyTorch ile aynıdır. Küçültürseniz bütün adımlar lineer olarak küçülür.
<code>rho</code>	0.9	Pencerelerin ne kadar geçmişe baktığı. RMSProp'taki alpha'nın karşılığıdır.
<code>eps</code>	1e-6	Hem pay hem payda içindedir. İlk adımda $E[\Delta^2] = 0$ olduğu için ilk Δ , ϵ sayesinde küçüktür.
<code>weight_decay</code>	0	Gradyana eklenir.

İki tampon vardır: kare gradyan penceresi ve adım penceresi. Öğrenme hızını ayarlamaktan kaçındığınız, ölçeği dengersiz kayıplarda kullanılır. Modern büyük modellerde nadiren AdamW'den iyidir.

10. Adam ve AdamW

İkisi aynı şablondur: `AdamImpl<T, false>` Adam, `AdamImpl<T, true>` AdamW. Fark tek bayraktır: weight decay momentten önce gradyana mı eklenir, yoksa parametreden ayrı mı düşülür.

Adam: $g \leftarrow g + \lambda p$ sonra momentler bu g 'yi görür.
AdamW: $p \leftarrow p - \gamma \lambda p$ momentler ham g 'yi görür. Decay, adaptif paydadadan geçmez.

$$\begin{aligned}m &\leftarrow \beta_1 m + (1-\beta_1) g \\ v &\leftarrow \beta_2 v + (1-\beta_2) g^2 \\ \hat{m} &= m / (1-\beta_1^t) \quad \hat{v} = v / (1-\beta_2^t) \\ p &\leftarrow p - \gamma \hat{m} / (\sqrt{\hat{v}} + \epsilon) \\ \text{amsgrad ise } \hat{v} &\text{nin kaynağı o ana kadarki en büyük } v \text{'dir.}\end{aligned}$$

Alan	Varsayılan	Ne işe yarar, yükseltince / düşürünce ne olur
<code>lr</code>	1e-3	Options kopyası. Asıl γ gruptadır. Dil modeli ince ayarında çoğu kez 1e-5...1e-4, sıfırdan MLP'de 1e-3 makuldür. Çok yüksekse loss sapar.
<code>beta1</code>	0.9	Birinci momentin (yönün) hafızası. 0.9, son yaklaşık 10 adımın gradyanına ağırlık verir. Yükseltmek (0.95) yönü yumuşatır, düşürmek güncel gradyana yapışır.
<code>beta2</code>	0.999	İkinci momentin (ölçeğin) hafızası. 0.999 uzun penceredir. Gürültülü küçük batch'te 0.99 denemek paydadaki zıplamayı azaltabilir. 1 olamaz; düzeltme paydası sıfırlanır.
<code>eps</code>	1e-8	Payda tabanı. Çok küçük ikinci momentte adımın sonsuza gitmesini keser. FP16 ince ayarda bazen 1e-6 kullanılır; bu kopyada varsayılan 1e-8'dir.
<code>weight_decay</code>	0	Adam'da gradyana karışır, yani büyük gradyanlı koordinat decay'i de küçük sanabilir. AdamW'de her koordinattan $\gamma \lambda p$ düşülür; 0.01 veya 0.1 klasiktir. Bias'a uygulamak genelde istenmez.
<code>amsgrad</code>	false	true ise ek tampon tutar ve paydada v 'nin tarihsel maksimumunu kullanır. Bazı yakınsama kanıtları için vardır. Çoğu görüntü ve dil eğitiminde kapalı kalır; bellek bir tampon daha yer.

İlk adımın elle hesabı

$p = 1, g = 0.1, \gamma = 1e-3, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1e-8, \lambda = 0, t = 1.$

- $m = 0.1 \cdot 0.1 = 0.01$ çünkü $(1-\beta_1) g$ ve $m_0 = 0.$
- $v = 0.001 \cdot 0.01 = 1e-5.$
- $1-\beta_1^1 = 0.1,$ dolayısıyla $\hat{m} = 0.01 / 0.1 = 0.1 = g.$

- $1 - \beta_2^1 = 0.001$, $\hat{v} = 1e-5 / 0.001 = 0.01 = g^2$.
- $\sqrt{\hat{v}} = 0.1$. Payda = $0.1 + 1e-8$.
- Adım = $0.001 \cdot 0.1 / 0.1 = 0.001$.
- $p \leftarrow 1 - 0.001 = 0.999$.

İlk adımda bias düzeltmesi momentleri ham gradyana çevirir. İşaret ile öğrenme hızının çarpımı kadar bir adım atılır. Bu, testteki kapalı formdur.

AdamW, $\lambda = 0.01$, aynı sayılar: önce $p \leftarrow 1 - 0.001 \cdot 0.01 \cdot 1 = 0.99999$. Momentler decay'siz $g = 0.1$ görür, yine 0.001 düşülür. Sonuç 0.99899. Decay ve adaptif adım birbirine karışmamıştır.

```
nexus_optim::AdamW<float>::Options opt;
opt.lr = 1e-3;
opt.beta1 = 0.9;
opt.beta2 = 0.999;
opt.eps = 1e-8;
opt.weight_decay = 0.01;
opt.amsgrad = false;

nexus_optim::ParamGroupOptions pg;
pg.learning_rate = 1e-3;

nexus_optim::AdamW<float> optimizer(
    {nexus_optim::make_group(p.data(), g.data(), n, pg)}, opt);

nexus_optim::CosineAnnealingLR sched(optimizer, /*T_max=*/1000);
optimizer.zero_grad();
// backward
nexus_optim::clip_grad_norm(optimizer.param_groups(), 1.0);
optimizer.step();
sched.step();
```

İki gruplu, decay'siz kafa:

```
nexus_optim::ParamGroupOptions head;
head.learning_rate = 1e-3;
head.weight_decay = 0;
head.weight_decay_override = true;
nexus_optim::AdamW<float> two(
    {nexus_optim::make_group(body, body_g, body_n, pg),
     nexus_optim::make_group(head_p, head_g, head_n, head)},
    opt);
```

11. NAdam, RAdam, AdaBelief

NAdam

Adam'ın birinci momentine Nesterov bakışı ekler. PyTorch 2.4 formülü uygulanır.

$$\mu_t = \beta_1 (1 - \frac{1}{2} \cdot 0.96^{t-\psi})$$

μ ürünü her adım μ_t ile çarpılır, başlangıç 1'dir.

m ve v olağan güncellenir.

$$p += (\text{adım}_g \cdot g + \text{adım}_m \cdot m) / (\sqrt{v / (1 - \beta_2^t)}) + \epsilon$$

Alan	Varsayılan	Ne işe yarar
lr	2e-3	Adam'dan biraz yüksek varsayılır; formülün ölçeği buna göre dir.
beta1, beta2, eps	0.9, 0.999, 1e-8	Adam ile aynı anlam.
momentum_decay ψ	4e-3	μ 'nün zamanla β_1 'den biraz aşağı kaymasını sağlar. Büyütünce Nesterov karışımı daha hızlı değişir. Varsayılanı değiştirmeyin, formül ona göre ayarlıdır.
weight_decay	0	Aşağıdaki bayrağa göre ya gradyana eklenir ya da parametreden düşülür.
decoupled_weight_decay	false	true ise NAdamW. Dil modellerinde decay kullanacaksanız true düşünün.

```
nexus_optim::NAdam<float>::Options o;
o.lr = 2e-3;
o.momentum_decay = 4e-3;
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 2e-3;
nexus_optim::NAdam<float> nadam({nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
```

RAdam

İlk adımlarda ikinci moment henüz güvenilir değildir. ρ_t 5'i geçene kadar güncelleme adaptif değildir:

$$\rho_\infty = 2/(1-\beta_2) - 1$$

$$\rho_t = \rho_\infty - 2 t \beta_2^t / (1 - \beta_2^t)$$

$$\rho_t \leq 5: p \leftarrow p - \gamma \cdot m / (1-\beta_1^t)$$

$$\rho_t > 5: \text{düzeltme çarpanı ile klasik adaptif adım}$$

Alan	Varsayılan	Ne işe yarar
lr	1e-3	Hem erken SGD benzeri fazın hem sonraki adaptif fazın ölçeği. Grup hızı zamanlayıcıyla değişince adaptif adım da grup hızından yeniden kurulur.
beta1, beta2, eps	0.9, 0.999, 1e-8	β_2 , ρ 'nün ne zaman 5'i aşacağını da belirler. Varsayılanla ilk birkaç adım düzeltmesiz gider.
weight_decay	0	decoupled_weight_decay false iken gradyana, true iken parametreye uygulanır.
decoupled_weight_decay	false	AdamW stili decay.

Ne zaman seçilir: Adam'ın ilk adımlarda loss'u sıçrattığı, ısınma yazmak istemediğiniz küçük deneyler. Büyük dil eğitiminde ısınmalı AdamW daha yaygındır.

AdaBelief

$$m \leftarrow \beta_1 m + (1-\beta_1) g$$

$$s \leftarrow \beta_2 s + (1-\beta_2) (g - m)^2 + \epsilon$$

$$p \leftarrow p - \gamma \hat{m} / (\sqrt{\hat{s}} + \epsilon)$$

"İnanç", gradyanın kendi ortalamasına ne kadar benzediğidir. Sapma küçükse (model öngördüğünü görüyorsa) payda küçülür, adım büyür. Sürpriz gradyanda adım küçülür.

Alan	Varsayılan	Ne işe yarar
lr	1e-3	Adım ölçeği.
beta1, beta2	0.9, 0.999	Ortalama ve sapma pencereleri.
eps	1e-8	Hem s'nin içine her adım eklenir hem paydaya. Bu, makaledeki sabitleyicidir. ε'u büyütmek s'yi şişirir ve adımı küçültür.
weight_decay	0	Decay miktarı.
decoupled_weight_decay	true	Varsayılan ayrık decay'dir. Kapamak için false yazın.

12. LAMB, LARS, Lion

LAMB — katman boyunu koruyan Adam

$u_0 = \hat{m} / (\sqrt{\hat{v}} + \epsilon)$
 $u = u_0 + \lambda p$
güven = $\|p\|_2 / \|u\|_2$ (pay veya payda 0 ise güven = 1)
 $p \leftarrow p - \gamma \cdot \text{güven} \cdot u$

Alan	Varsayılan	Ne işe yarar
lr	1e-3	Büyük batch'te Adam'a göre yüksek hız kaldıracığı için tercih edilir. Yine de LinearWarmup ile açın.
beta1, beta2	0.9, 0.999	Adam ile aynı momentler. Güven oranından önce hesaplanır.
eps	1e-6	Adam'dan büyük varsayılan. Paydadaki küçük v'lerin güven oranını şişirmesini keser.
weight_decay	0	u'nun içine λp olarak girer, yani hem yönde hem $\ u\ $ normunda vardır. 0.01 tipiktir. Güven oranını da etkiler; "önce decay sonra güven" sırası budur.

Ek bellek: m, v ve bir scratch u. Scratch checkpoint'e yazılmaz. Güven katman tensörünün tamamına göredir; 50.000'den büyük diye parçalara bölünüp her parçaya ayrı güven verilmez. CUDA'da bu norm atomik blok toplamıyla gelir, sıra bit-tam aynı olmayabilir, test toleransı 2e-4'tür.

```

nexus_optim::LAMB<float>::Options o;
o.lr = 1e-3; o.weight_decay = 0.01;
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 1e-3;
nexus_optim::LAMB<float> lamb({nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
nexus_optim::LinearWarmup warm(lamb, 1000);

```

LARS — aynı fikir, SGD tabanı

güven = $\eta \|p\| / (\|g\| + \lambda \|p\| + \epsilon)$
ölçek = $g + \lambda p$
 $\mu \neq 0$ ise $v \leftarrow \mu v + \text{ölçek}$, aksi halde v yerine ölçek
 $p \leftarrow p - \gamma \cdot \text{güven} \cdot v$

Alan	Varsayılan	Ne işe yarar
lr	1	Güven zaten katman ölçeğini içeri alır. 1, orijinal tarifteki gibi yüksek bir bazdır. Küçük modellere körlemesine koymayın.
momentum	0.9	0 yaparsanız hız tamponu hiç ayrılmaz.
weight_decay	0	Hem güncellemeye eklenir hem paydadaki $\ g\ + \lambda\ p\ $ terimine girer. Güveni küçültür.
eta (η)	1e-3	Güven oranının çarpanı. Asıl "ne kadar cesur" sorusunun cevabı η'dir. lr'den bağımsız küçültmek bütün katmanları sakinleştirir.
eps	1e-8	Sıfır norma karşı.

Lion — işaret ve tek tampon

$$c = \beta_1 m + (1 - \beta_1) g$$
$$p \leftarrow (1 - \gamma \lambda) p - \gamma \cdot \text{sign}(c)$$
$$m \leftarrow \beta_2 m + (1 - \beta_2) g$$
$$\text{sign}(0) = 0$$

Alan	Varsayılan	Ne işe yarar
lr	1e-4	Adam'dan küçük tutulur çünkü adımın büyüklüğü doğrudan γ 'dır; payda yoktur. 1e-4 civarı başlayın. 1e-3 çoğu ağda serttir.
beta1	0.9	İşaretin bakacağı karışım: ne kadar eski m, ne kadar taze g. Yönü belirler, büyüklüğü değil.
beta2	0.99	m'nin kendisi bu katsayıyla güncellenir. β_1 'den ayırdır. 0.99, işarettten daha uzun bir hafızadır.
weight_decay	0	Her zaman ayrıktır: çarpan $(1 - \gamma \lambda)$. AdamW'dekiyle aynı fikirdir. 0.01...1 aralığı Lion yazınlarında kullanılır; büyük λ parametreyi hızlı söndürür.

Bellek Adam'ın yarısıdır: yalnız m. Sayı: $p = 0.5$, $g = 1$, $\gamma = 1e-3$, $\lambda = 0$. Karışım = $0.1 \cdot 1 = 0.1$, $\text{sign} = +1$, $p = 0.5 - 0.001 = 0.499$. Gürültüye karşı büyüklük üretmediği için batch gürültüsünde bazen AdamW'den sakın durur; lr'yi Adam'dan devralırsanız ıraksar.

```
nexus_optim::Lion<float>::Options o;  
o.lr = 1e-4; o.beta1 = 0.9; o.beta2 = 0.99; o.weight_decay = 0.01;  
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 1e-4;  
nexus_optim::Lion<float> lion({nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
```

13. Lookahead

Herhangi bir optimizier'in üstüne oturan yavaş ağırlık katmanıdır. İç optimizier her adım hızlı ağırlığı (sizin parametrenizi) günceller. Her k adımda bir yavaş kopya hızlıya doğru α kadar yürür ve hızlı, yavaşla eşitlenir.

$$\text{yavaş} \leftarrow (1 - \alpha) \text{yavaş} + \alpha \text{hızlı}$$
$$\text{hızlı} \leftarrow \text{yavaş}$$

Argüman	Anlam
k	Kaç iç adımda bir karışım. 1 ise her adım. 5 veya 10 yaygındır. 1'den küçükse kurucu hata verir.
alpha	Yavaşın hızlıya ne kadar inandığı. 0.5 yarı yarıyadır. 1 hızlıyı olduğu gibi yavaş beller. (0, 1] dışında hata.

Yavaş kopya kurucuda parametrenin o anki değeridir. Ek bellek, parametre boyutunun bir kopyasıdır. Sayı: $k = 1$, $\alpha = 0.5$, SGD $\gamma = 0.1$, $p = 1$, $g = 1$. İç adım p'yi 0.9 yapar. Yavaş 1 ile karışır: $0.5 \cdot 1 + 0.5 \cdot 0.9 = 0.95$. Parametre 0.95'e çekilir.

```
nexus_optim::SGD<float> inner({group}, sgd_opt);  
nexus_optim::Lookahead<nexus_optim::SGD<float>> opt(std::move(inner), 6, 0.5f);  
opt.zero_grad();  
opt.step();  
opt.param_groups(); // zamanlayıcı bağlamak için  
opt.base(); // alttaki SGD
```

14. Zamanlayıcıların her argümanı

Hepsi kurucuda optimizier referansı alır ve `param_groups()` içindeki hızların adresini kaydeder. Baz hız, kurucu anındaki gruptur. `last_epoch` -1'den başlar; ilk `step()` onu 0 yapar. `get_last_lr()` birinci grubun son hızı, `get_last_lrs()` hepsidir.

StepLR(opt, step_size, gamma = 0.1)

Formül: $\text{baz} \cdot \gamma^{\lfloor \text{epoch} / \text{step_size} \rfloor}$. step_size en az 1. Örnek: baz 0.8, step_size 2, gamma 0.1. Adım 0 ve 1 hâlâ 0.8'dir. Adım 2'de 0.08 olur. "Her iki epoch'ta bir" sayıyorsanız ilk düşüş üçüncü scheduler.step() çağrısındadır, çünkü sayaç 0'dan başlar.

ExponentialLR(opt, gamma)

$\text{baz} \cdot \gamma^{\text{epoch}}$. gamma 0.5 ise dizi 1, 0.5, 0.25, ... katlarıdır (baz 1 iken). 1'den büyük gamma hızı büyütür. 0 olursa ikinci adımdan itibaren hız 0'dır.

CosineAnnealingLR(opt, T_max, eta_min = 0)

$\eta_{\min} + (\text{baz} - \eta_{\min}) \cdot (1 + \cos(\pi t / T_{\max})) / 2$. t, T_max'ı aşarsa T_max'ta tutulur. T_max en az 1. T_max = 2, eta_min = 0, baz 1: epoch 0'da 1, epoch 1'de 0.5, epoch 2'de 0. Eğitim uzunluğu kadar T_max verin; kısa T_max hızı erken tabana yapıştırır.

CosineAnnealingWarmRestarts(opt, T_0, T_mult = 1, eta_min = 0)

SGDR. Periyot T_i = T_0 ile başlar. T_cur, T_i olunca sıfırlanır ve T_i, T_mult ile çarpılır. T_mult = 1 ise periyotlar eşit kalır, hız her periyotta bazdan geri açılır. T_mult = 2 periyotları uzatır. T_0 ve T_mult en az 1.

OneCycleLR(opt, max_lr, total_steps, pct_start = 0.3, div_factor = 25, final_div_factor = 1e4)

Argüman	Ne işe yarar
max_lr	Çevrimin tepesi. Grubun baz hızı ile aynı olmak zorunda değildir; çevrim bu sayıdan kurulur, grupların birbirine oranı birinci grubun bazına bölünerek korunur.
total_steps	Çevrimin uzunluğu, eğitim adımı cinsinden. En az 1. Bitince hız tabanda kalır.
pct_start	Yükselişin çevrime oranı. 0.3 ise ilk %30 doğrusal tırmanış, kalanı kosinüs iniş.
div_factor	Başlangıç = max_lr / div_factor. 25 ise tepeye 25 kat küçük başlar.
final_div_factor	Bitiş = max_lr / (div_factor · final_div_factor). 1e4 ile bitiş, başlangıcın 10 000'de biri kadar küçüktür.

LinearWarmup(opt, warmup_steps)

Hız = $\text{baz} \cdot \min(1, (\text{epoch}+1) / \text{warmup_steps})$. warmup_steps = 4, baz = 0.4 ise çağrılar 0.1, 0.2, 0.3, 0.4 üretir. Büyük batch ve LAMB ile ilk bin adıma koyun. Isınma bitince bu nesneyi kullanmayı bırakın; yoksa hız bazda kalır, düşmez. Düşüş için yeni bir kosinüs kurun. İkisini aynı adımda üst üste çağırırsanız ikisi de kendi bazına kilitlendiği için birbirini ezer.

ReduceLROnPlateau(opt, factor = 0.1, patience = 10, threshold = 1e-4, cooldown = 0, min_lr = 0)

Daha küçük metrik daha iyidir. İyileşme, $\text{yeni} < \text{en_iyi} \cdot (1 - \text{threshold})$ demektir. threshold 1e-4 ise yüzde 0.01'den küçük kıpırdamalar iyileşme sayılmaz. patience kadar kötü gözlemden sonra hız factor ile çarpılır. factor (0, 1) aralığında olmalıdır; 0.1 hızı onda bire indirir. İndirimden sonra cooldown kadar adım yok sayılır. min_lr tabandır. Metriksiz step() hata fırlatır.

```
nexus_optim::ReduceLROnPlateau plateau(opt, /*factor=*/0.1, /*patience=*/5);  
// doğrulama sonunda:  
plateau.step(val_loss);
```

15. Gradyan fonksiyonları

Bunlar parametreyi değil gradyanı değiştirir ve step'ten önce çağrılır. CUDA gradyanını bugün atlarlar; kırpma host bellek içindir.

clip_grad_norm_(groups, max_norm, norm_type = 2)

Dönüş: kırpmadan önceki norm. p = 2 varsayılandır ve SIMD kare toplam kullanır. p = sonsuz ise elemanların en büyük mutlak değeri alınır. Başka p için eleman bazlı pow vardır, yavaştır. max_norm negatifse hata. Ölçek = $\min(1, \text{max_norm} /$

(norm + 1e-6)). 1e-6, sıfır gradyanda bölmeyi keser ve PyTorch ile aynıdır. Örnek: $g = (3, 4, 0)$, max 1, norm 5, $g \approx (0.6, 0.8, 0)$.

Ne işe yarar: bir batch'in patolojik gradyanı bütün adımı uçurmasın. max_norm 1, dil modellerinde sık kullanılan emniyettir. Çok küçük max_norm öğrenmeyi boğar; çok büyükse hiç kırpamazsınız.

clip_grad_value_(groups, clip)

Her elemanı $[-|clip|, |clip|]$ aralığına keser. Norm kırpmasından farkı: yönü küresel olarak ölçeklemez, uç elemanı tek tek keser. İkisi birden nadiren gerekir. Biri yetiyorsa norm kırpması daha yaygındır.

scale_gradients_(groups, scale)

Gradyanı skalerle çarpar. Biriktirmenin standart kullanımı: N mikro-batch'te zero_grad yok, her biri geri yayılım yapıyor, sonra

```
nexus_optim::scale_gradients_(opt.param_groups(), 1.0 / N);
opt.step();
opt.zero_grad();
```

Bunu unutursanız etkin öğrenme hızınız N katına çıkar.

gradient_centralization_(groups, channels = 0)

channels 0 ise tensör tek vektördür, ortalaması çıkarılır. Aksi halde eleman sayısı kanala bölünmek zorundadır; bölünmezse hata. Kalan eksen 1 ise (bias) fonksiyon o tensöre dokunmaz, çünkü çıkarılacak bir yayılım yoktur. CNN ve lineer katmanın ağırlık gradyanında, kanal başına ortalamayı silmek bazı mimarilerde yakınsamayı yumuşatır. Kırpmadan önce uygulayın ki norm, merkezlenmiş gradyanı görsün.

16. FP16, BF16, CUDA, MatrixFlash

Dar depolama

float16::from_float / to_float IEEE FP16 yuvarlamasıdır. bfloat16 16 bitlik üst yarıyı, en yakın çifte yuvarlayarak keser. İkisi de 2 bayt hizalıdır. Hesap FP32 master parametrede ve FP32 momentte yürür. Master ve çevrilmiş gradyan tamponu kurucuda ayrılır. Adım sonunda master, çağıranın dar tamponuna geri yazılır. CUDA bu tipleri kabul etmez; cihaz yolu FP32'dir.

```
std::vector<nexus_optim::float16> p(n), g(n);
p[i] = nexus_optim::float16::from_float(0.5f);
g[i] = nexus_optim::float16::from_float(0.01f);
nexus_optim::AdamW<nexus_optim::float16>::Options o;
o.lr = 1e-3; o.weight_decay = 0.01;
nexus_optim::ParamGroupOptions pg; pg.learning_rate = 1e-3;
nexus_optim::AdamW<nexus_optim::float16> opt(
    {nexus_optim::make_group(p.data(), g.data(), n, pg)}, o);
opt.step();
float readable = p[0].to_float();
```

CUDA'yı kendiniz bağlamak

```
float *d_p = nullptr, *d_g = nullptr;
cudaMalloc(&d_p, n * sizeof(float));
cudaMalloc(&d_g, n * sizeof(float));
cudaMemcpy(d_p, host_p, n * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_g, host_g, n * sizeof(float), cudaMemcpyHostToDevice);

nexus_optim::ParamGroup<float> g;
g.params = {d_p}; g.grads = {d_g}; g.numels = {n};
g.devices = {nexus_optim::Device::CUDA};
g.options.learning_rate = 1e-3;

nexus_optim::AdamW<float>::Options o;
o.lr = 1e-3; o.weight_decay = 0.01;
nexus_optim::AdamW<float> gpu({g}, o);
gpu.zero_grad(); // cihaz gradyanını sıfırlar; sonra backward d_g'yi doldurmalı
gpu.step();      // dönünce d_p günceldir
cudaMemcpy(host_p, d_p, n * sizeof(float), cudaMemcpyDeviceToHost);
```

Momentleri siz ayırmazsınız. Kurucu, her CUDA tensörü için cihaz tamponu ister. `double` veya `float16` ile `Device::CUDA` hata verir. CUDA kapalı derlemede herhangi bir CUDA tensörü hata verir.

MatrixFlash

Şemsiye başlık adaptörü içermez. Ayrıca şunu ekleyin ve CMake'e include yolunu verin:

```
#include <nexus_optim/adapters/matrixflash.hpp>

auto group = nexus_optim::group_from_host(params, grads, options);
// cihaz eğitiminde:
auto dev = nexus_optim::group_from_device(params, grads, options);
opt.step();
nexus_optim::mark_host_stale(params);
```

Gerçek Tensor API'si varsayılandan farklıdır. Tip `matrix_pro::Tensor`, şablon değil, eleman `float`. Uzunluk `size()`. Host ayna `host_ptr()`. Cihaz `device_data()`. `group_from_host` indirme çağırır; aynanın güncel olması sizin işiniz. `group_from_device` işaretçi yoksa hata verir. Adaptörden sonra host'tan okuyacaksınız `download()` veya taze ayna gerekir; `mark_host_stale` aynanın bayat olduğunu işaretler.

Durum kaydı

```
auto snap = opt.state_dict();
// snap.algorithm, snap.step, snap.group_learning_rates, snap.blobs, snap.scalars
// Parametreleri model kaydı tutsun. Burada yalnızca moment vardır.

nexus_optim::AdamW<float> resumed(groups_same_shapes, same_options);
resumed.load_state_dict(snap);
```

Yüklemeden önce parametrelerin kendisi de checkpoint değerine getirilmiş olmalıdır. Aksi halde moment yeni parametreyle uyumsuz ve ilk adım saçmalar. Blob bayt sayısı `numel * sizeof(float)` olmalıdır. LAMB scratch'i kayda girmez. NAdam'ın μ çarpımı `scalars` içindedir.

17. Hazır reçeteler

Küçük ağ, sıfırdan, AdamW

lr 1e-3, weight decay 0.01, beta varsayılan, clip 1, kosinüs T_max = toplam adım. Bias varsa ikinci grupta override ile decay 0.

Büyük batch, LAMB

lr 1e-3 veya batch büyüdükçe yukarı, weight decay 0.01, eps 1e-6, LinearWarmup 1000 adım, sonra ayrı bir CosineAnnealingLR. Aynı adımda ikisini birden adım atmayın.

İşaret tabanlı, az bellek

Lion, lr 1e-4 (Adam'daki 1e-3'ü kopyalamayın), beta2 0.99, decay 0.01. Bellek Adam'ın yaklaşık yarısıdır.

İlk adımlar zıplıyor, ısınma yazmak istemiyorum

RAdam, lr 1e-3. ρ 5'in altındayken adaptif payda kapalıdır.

Mikro-batch sığmıyor

N adım boyunca `zero_grad` yok. Sonra `scale_gradients(groups, 1.0/N)`, bir `step`, bir `zero_grad`. Zamanlayıcıyı her mikro-batch'te değil, bu gerçek adımda bir ilerletin.

Doğrulama loss'u takıldı

`ReduceLROnPlateau`, factor 0.1, patience 5 veya 10. Eğitim hızı zamanlayıcısından ayrı tutun; ikisi aynı grubu yazarsa son çağıran kazanır.

18. Hatalar ve kontrol listesi

Belirti	Sebebi
Kurucu <code>invalid_argument</code> , "at least one"	Grup listesi boş.
"same length"	params, grads, numels, devices boyutları tutmuyor.
"null data pointer"	Uzunluğu pozitif tensörün işaretçisi boş.
Parametre hiç değişmiyor	<code>zero_grad</code> sonrası gradyanı yeniden yazmadınız, veya γ 0, veya yanlış vektöre backward yazıyorsunuz.
Loss bir anda NaN	γ büyük, clip yok, veya gradyan zaten Inf. NaN, adımı sessizce atlamaz; yayılır.
Decay bias'ta da var	<code>weight_decay_override</code> açılmadı. Varsayılan false, optimizier λ 'sını kullanır.
Zamanlayıcı hızı değiştirmiyor gibi	StepLR'de ilk düşüş <code>step_size</code> 'inci değil, sayaç 0'dan başladığı için bir sonraki kattadır. Ayrıca <code>param_groups</code> vektörünü büyüttüyseniz adres düşmüştür.
İki zamanlayıcı kavga ediyor	Aynı adımda ikisi de bazdan hesaplayıp üstüne yazar. Birini bırakın.
CUDA runtime_error	Derleme bayrağı kapalı, veya skaler double/float16, veya <code>device_data()</code> boş.
State yüklenmiyor	Algoritma adı uyuşmuyor veya tensör şekli değişti.
Resume sonrası ilk adım saçma	Moment yüklendi, parametre checkpoint'e çekilmedi.
GPU, CPU'dan yavaş	1e6 civarı elemanda senkron maliyeti çekirdeği ezer. Parametre cihazda ve daha büyükse yeniden ölçün.

Kontrol listesi, her yeni eğitimden önce: grup γ 'sı Options γ 'sı ile aynı mı, decay'siz gruplarda override var mı, `zero_grad` biriktirmenin neresinde, clip `step`'ten önce mi, zamanlayıcı `step`'ten sonra mı, CUDA ise işaretçiler cihaz mı, checkpoint'te hem parametre hem `state_dict` var mı.

NexusOptim 1.0 ayrıntılı kılavuz. Formüller, varsayılanlar ve süreler bu deponun kaynak kodu ve Release ölçümleriyle aynıdır. AVX-512 bu CPU'da çalışmadığı için o sütun boştur. GPU ölçümü stream senkronu içerir.